



Round Robin vs Preemptive Priority Scheduling

Technical Report | Algorithm Comparison Project
Operating Systems -1

Compared Algorithms

Round Robin - Preemptive Priority Scheduling

Focus: Fairness vs Urgency | Quantum & Priority inputs | Five test scenarios

1. Introduction

This report documents the design, implementation, and test results of the Round Robin vs Preemptive Priority Scheduling comparison project, submitted as part of the Operating Systems -1 algorithm comparison assignment. The project compares two fundamentally different CPU scheduling philosophies: time-sharing fairness (Round Robin) against urgency-driven preemption (Priority Scheduling).

The simulator is implemented in Java using Swing for the graphical interface (GUI). It accepts dynamic process input, validates all fields, runs both algorithms on the same workload, and presents side-by-side Gantt charts, per-process metrics, averages, and a written comparison. A console mode is also available for command-line execution.

1.1 Algorithms Overview

Round Robin (RR)

Round Robin assigns each ready process a fixed CPU slice called the time quantum. When a process uses its full quantum without completing, it is moved to the back of the ready queue so the next process can run. This rotation continues until all processes are finished. The key properties are:

- Every process receives CPU time at regular intervals regardless of burst length or priority.
- Response time is bounded: no process waits longer than $(n-1) \times \text{quantum}$ before its first turn.
- A small quantum increases context-switch overhead; a large quantum converges toward FCFS behavior.

Preemptive Priority Scheduling

Preemptive Priority Scheduling always runs the highest-priority process among those that have already arrived. If a higher-priority process arrives while a lower-priority one is running, the running process is immediately preempted. Tie-breaking rules applied in this implementation are:

- Primary: lower priority number = higher priority (configurable).
- Secondary tie: earlier arrival time wins.
- Tertiary tie: smaller process ID wins.

1.2 Metrics Definitions

Metric	Formula	Meaning
Turnaround Time (TAT)	Completion Time – Arrival Time / Burst Time + Waiting Time	Total elapsed time from arrival to finish
Waiting Time (WT)	TAT – Burst Time	Time spent waiting in the ready queue
Response Time (RT)	Process Gets CPU – Arrival Time	Time until process first receives the CPU

2. Implementation Notes

2.1 Project Structure

The project follows a layered package structure separating concerns cleanly:

- scheduler.Model — Process, GanttEntry, ScheduleResult data classes.
- scheduler.Algorithm — Scheduler interface, RoundRobinScheduler, PriorityScheduler.
- scheduler.Metrics — MetricsCalculator (WT, TAT, RT, averages).
- scheduler.Validation — InputValidator with individual check methods.
- scheduler.Gui — MainWindow, SimulationPanel, ComparisonPanel, GanttChartPanel, MetricsTablePanel, ProcessTableModel.

2.2 Algorithm Assumptions

- Priority scheduling is preemptive: a running process is interrupted immediately when a higher-priority process arrives.
- Round Robin uses a configurable quantum (default 3); arrivals mid-quantum are added to the queue and considered at the next scheduling point.
- Both algorithms use the same deep copy of the input process list so neither modifies the original data.
- Idle CPU time is recorded as GanttEntry with processId = -1 and rendered as a grey block.
- Priority rule (lower Number = higher priority or higher number = higher priority) is configurable from the GUI and applies to both comparison runs.

2.3 Known Limitations

- Starvation prevention (ageing) is not implemented for Priority Scheduling. Scenario D demonstrates the observable consequence.
- The Gantt chart panel scales block width to fit the window; very long simulations may compress labels. A horizontal scroll pane is used to mitigate this.
- The project requires Java 21 and NetBeans or any IDE supports Java; no external libraries are used beyond standard Java Swing.

3. Test Scenarios and Results

The same workload is used for both algorithms in every scenario. Results below are computed by running the implemented simulator. Gantt chart labels show which process held the CPU in each time slice; time markers appear below each block.

Scenario A — Basic Mixed Workload

Input Processes | Time Quantum = 3

Process	Arrival Time	Burst Time	Priority	Rule: lower # = higher priority
P1	0	7	3	
P2	2	4	1	
P3	4	1	4	
P4	5	4	2	
P5	6	3	5	

Gantt Charts

Round Robin (Q = 3)

P1	P2	P1	P3	P4	P5	P2	P1	P4	
0	3	6	9	10	13	16	17	18	19

Preemptive Priority Scheduling

P1	P2	P4	P1	P3	P5		
0	2	6	10	15	16	19	

Metrics Table (RR = Round Robin | Pr = Priority)

Process	AT	BT	Priority	RR WT	RR TAT	RR RT	Pr WT	Pr TAT
P1	0	7	3	11	18	0	8	15
P2	2	4	1	11	15	1	0	4
P3	4	1	4	5	6	5	11	12
P4	5	4	2	10	14	5	1	5
P5	6	3	5	7	10	7	10	13
Averages				8.8	12.6	3.6	6	9.8

Comparison Summary

Metric	Round Robin	Priority	Winner
Average Waiting Time	8.8	6	Priority

Average Turnaround Time	12.6	9.8	Priority
Average Response Time	3.6	4.4	Round Robin

Analysis

- 1) Which algorithm gave better average waiting time?
-> Priority (RR=8.80, Priority=6.00)
- 2) Which algorithm gave better response time?
-> Round Robin (RR=3.60, Priority=4.40)
- 3) Did higher-priority processes gain significant advantage?
-> Yes, priority-sensitive behavior is visible.
- 4) Did Round Robin appear more balanced across all processes?
-> Yes, it generally rotates CPU slices more evenly.
- 5) Was starvation observed or likely in Priority Scheduling?
-> Not clearly observed in this run.
- 6) Which algorithm would you recommend for the tested workload, and why?
-> Preemptive Priority because it gives a better combined WT/RT trade-off for this dataset of 5 processes.

CPU Scheduling Comparator — Round Robin vs Priority

SimulationComparison

Time Quantum: 3Priority Rule: ☒ Lower number = higher priority ☐ Higher number = higher priorityPreset: Scenario A - Basic mixed workloadLoad Scenario

Add ProcessQuick PasteDelete ItemClear AllRun SimulationReset

Process Input Table

Process ID	Arrival Time	Burst Time	Priority
	1	0	7
	2	2	4
	3	4	1
	4	5	4
	5	6	3

Round Robin | Quantum = 3

P1P2P1P3P4P5P2P1P4

0369101316171819

Queue: P1 -> P2 -> P1 -> P3 -> P4 -> P5 -> P2 -> P1 -> P4

Process	Arrival	Burst	Priority	WT	TAT	RT
P1	0	7	3	11	18	0
P2	2	4	1	11	15	1
P3	4	1	4	5	6	5
P4	5	4	2	10	14	5
P5	6	3	5	7	10	7

Averages: WT = 8.80 TAT = 12.60 RT = 3.60

Avg WT: 8.80 | Avg TAT: 12.60 | Avg RT: 3.60

Preemptive Priority Scheduling

P1P1P2P2P2P2P4P4P4P4P1P1P1P1P

01234567891011121314

Queue: P1 -> P2 -> P4 -> P1 -> P3 -> P5

Process	Arrival	Burst	Priority	WT	TAT	RT
P1	0	7	3	8	15	0
P2	2	4	1	0	4	0
P3	4	1	4	11	12	11
P4	5	4	2	1	5	1
P5	6	3	5	10	13	10

Averages: WT = 6.00 TAT = 9.80 RT = 4.40

Avg WT: 6.00 | Avg TAT: 9.80 | Avg RT: 4.40

Operating Systems-1— Algorithm Comparison Project

Page ...

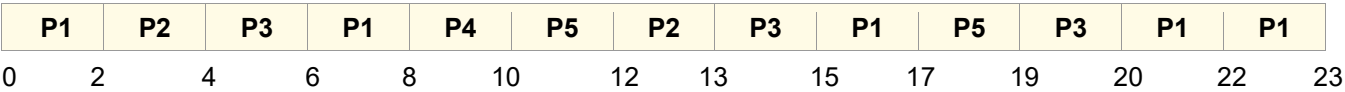
Scenario B — Urgency Case

Input Processes | Time Quantum = 2

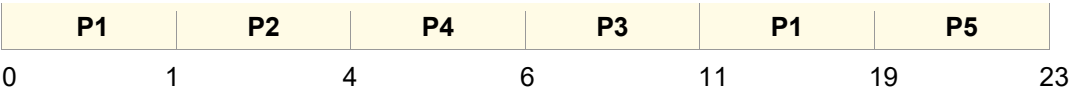
Process	Arrival Time	Burst Time	Priority	Rule: lower # = higher priority
P1	0	9	4	
P2	1	3	1	
P3	2	5	3	
P4	3	2	2	
P5	4	4	5	

Gantt Charts

Round Robin (Q = 2)



Preemptive Priority Scheduling



Metrics Table (RR = Round Robin | Pr = Priority)

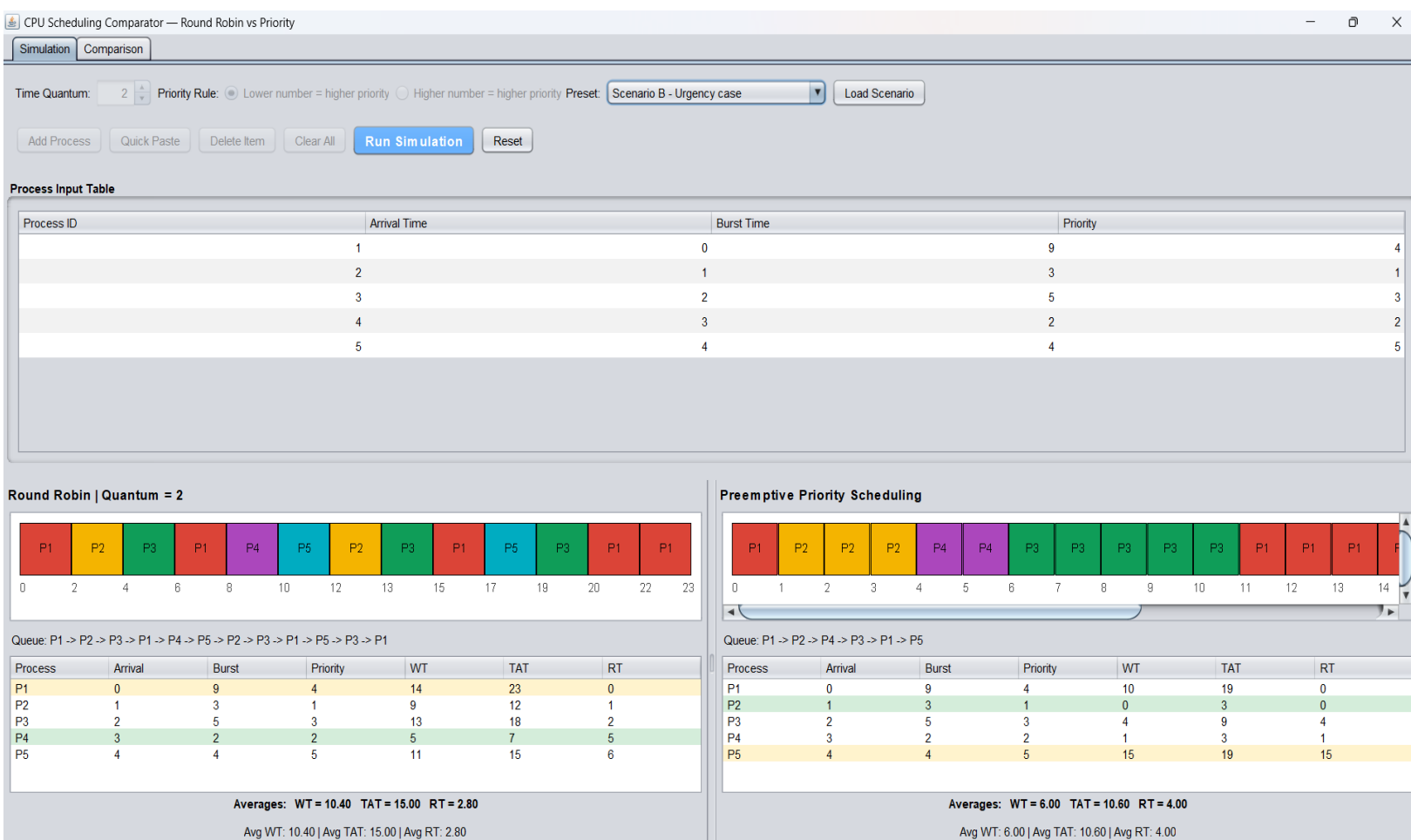
Process	AT	BT	Priority	RR WT	RR TAT	RR RT	Pr WT	Pr TAT
P1	0	9	4	14	23	0	10	19
P2	1	3	1	9	12	1	0	3
P3	2	5	3	13	18	2	4	9
P4	3	2	2	5	7	5	1	3
P5	4	4	5	11	15	6	15	19
Averages				10.4	15	2.8	6	10.6

Comparison Summary

Metric	Round Robin	Priority	Winner
Average Waiting Time	10.4	6	Priority
Average Turnaround Time	15	10.6	Priority
Average Response Time	2.8	4	Round Robin

Analysis

- 1) Which algorithm gave better average waiting time?
-> Priority (RR=10.40, Priority=6.00)
- 2) Which algorithm gave better response time?
-> Round Robin (RR=2.80, Priority=4.00)
- 3) Did higher-priority processes gain significant advantage?
-> Yes, priority-sensitive behavior is visible.
- 4) Did Round Robin appear more balanced across all processes?
-> Yes, it generally rotates CPU slices more evenly.
- 5) Was starvation observed or likely in Priority Scheduling?
-> Likely.
- 6) Which algorithm would you recommend for the tested workload, and why?
-> Preemptive Priority because it gives a better combined WT/RT trade-off for this dataset of 5 processes.



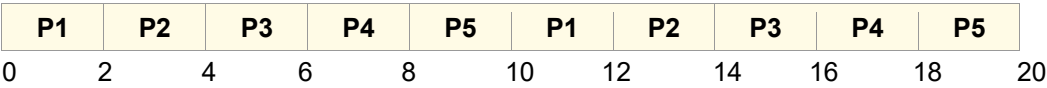
Scenario C — Fairness Case

Input Processes | Time Quantum = 2

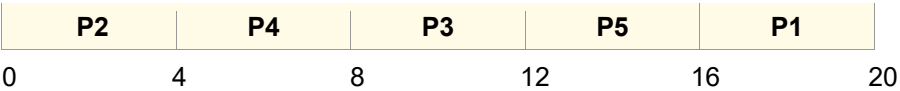
Process	Arrival Time	Burst Time	Priority	Rule: lower # = higher priority
P1	0	4	5	
P2	0	4	1	
P3	0	4	3	
P4	0	4	2	
P5	0	4	4	

Gantt Charts

Round Robin (Q = 2)



Preemptive Priority Scheduling



Metrics Table (RR = Round Robin | Pr = Priority)

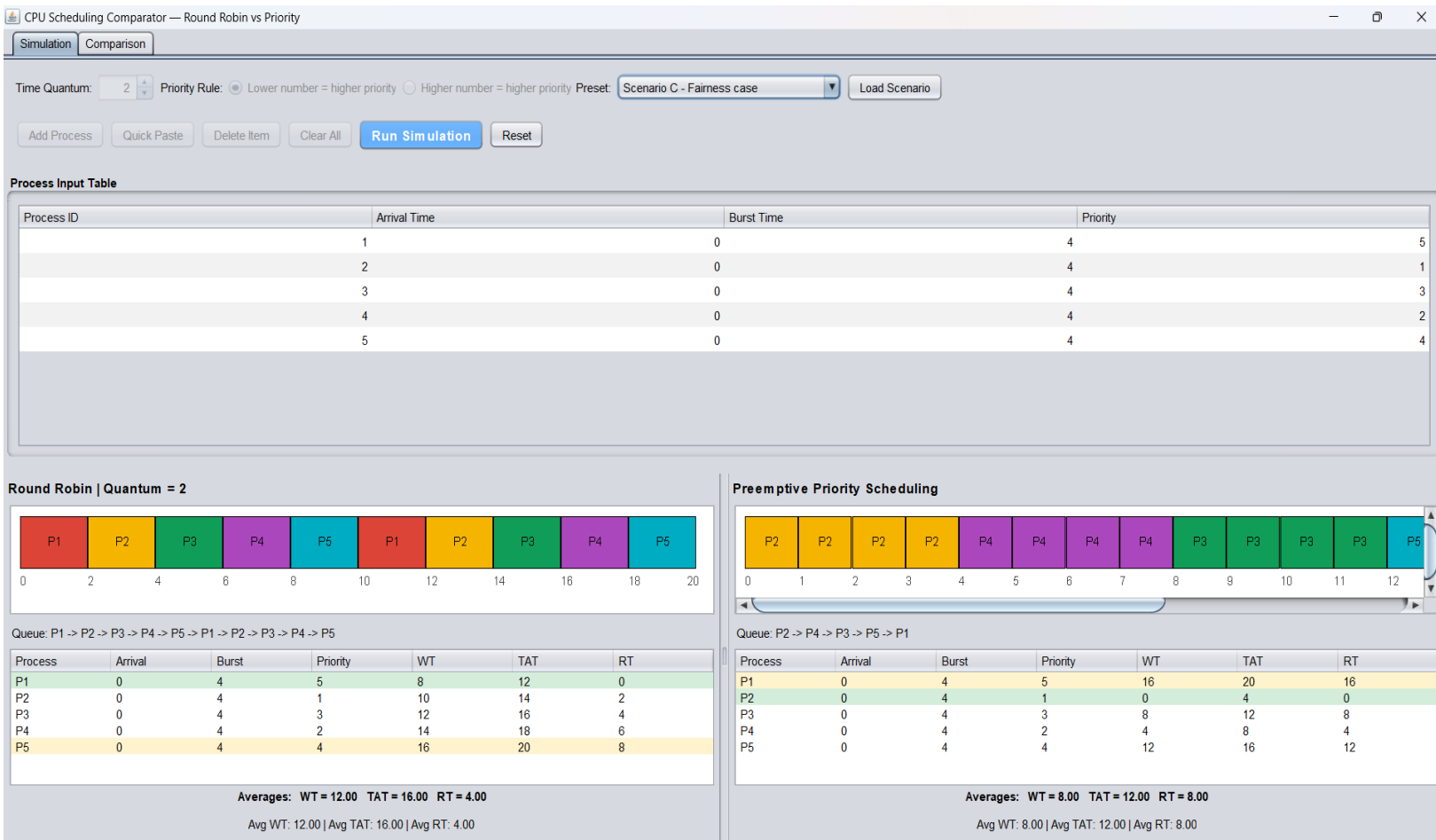
Process	AT	BT	Priority	RR WT	RR TAT	RR RT	Pr WT	Pr TAT
P1	0	4	5	8	12	0	16	20
P2	0	4	1	10	14	2	0	4
P3	0	4	3	12	16	4	8	12
P4	0	4	2	14	18	6	4	8
P5	0	4	4	16	20	8	12	16
Averages				12	16	4	8	12

Comparison Summary

Metric	Round Robin	Priority	Winner
Average Waiting Time	12	8	Priority
Average Turnaround Time	16	12	Priority
Average Response Time	4	8	Round Robin

Analysis

- 1) Which algorithm gave better average waiting time?
-> Priority (RR=12.00, Priority=8.00)
- 2) Which algorithm gave better response time?
-> Round Robin (RR=4.00, Priority=8.00)
- 3) Did higher-priority processes gain significant advantage?
-> Not strongly in this run.
- 4) Did Round Robin appear more balanced across all processes?
-> Yes, it generally rotates CPU slices more evenly.
- 5) Was starvation observed or likely in Priority Scheduling?
-> Not clearly observed in this run.
- 6) Which algorithm would you recommend for the tested workload, and why?
-> Tie (depends on fairness vs urgency preference) because it gives a better combined WT/RT trade-off for this dataset of 5 processes.



Scenario D — Starvation Risk

Input Processes | Time Quantum = 2

Process	Arrival Time	Burst Time	Priority	Rule: lower # = higher priority
P1	0	25	10	
P2	1	2	1	
P3	2	2	1	
P4	3	2	1	
P5	4	2	1	
P6	5	2	1	
P7	6	2	1	
P8	7	2	1	

Gantt Charts

Round Robin (Q = 2)

P1	P2	P3	P1	P4	P5	P6	P7	P8	P1	
0	2	4	6	8	10	12	14	16	18	39

Preemptive Priority Scheduling

P1	P2	P3	P4	P5	P6	P7	P8	P1	
0	1	3	5	7	9	11	13	15	39

Metrics Table (RR = Round Robin | Pr = Priority)

Process	AT	BT	Priority	RR WT	RR TAT	RR RT	Pr WT	Pr TAT
P1	0	25	10	14	39	0	14	39
P2	1	2	1	1	3	1	0	2
P3	2	2	1	2	4	2	1	3
P4	3	2	1	5	7	5	2	4
P5	4	2	1	6	8	6	3	5
P6	5	2	1	7	9	7	4	6
P7	6	2	1	8	10	8	5	7
P8	7	2	1	9	11	9	6	8
Averages				6.5	11.38	4.75	4.38	9.25

Comparison Summary

Metric	Round Robin	Priority	Winner
Average Waiting Time	6.5	4.38	Priority
Average Turnaround Time	11.38	9.25	Priority
Average Response Time	4.75	2.63	Priority

Analysis

- 1) Which algorithm gave better average waiting time?
-> Priority (RR=6.50, Priority=4.38)
- 2) Which algorithm gave better response time?
-> Priority (RR=4.75, Priority=2.63)
- 3) Did higher-priority processes gain significant advantage?
-> Yes, priority-sensitive behavior is visible.
- 4) Did Round Robin appear more balanced across all processes?
-> Yes, it generally rotates CPU slices more evenly.
- 5) Was starvation observed or likely in Priority Scheduling?
-> observed .
- 6) Which algorithm would you recommend for the tested workload, and why?
-> Preemptive Priority because it gives a better combined WT/RT trade-off for this dataset of 8 processes.

CPU Scheduling Comparator — Round Robin vs Priority

Simulation

Comparison

Time Quantum: 2

Priority Rule: ☒ Lower number = higher priority ☐ Higher number = higher priority

Preset: Scenario D - Starvation case

Load Scenario

Add Process

Quick Paste

Delete Item

Clear All

Run Simulation

Reset

Process Input Table

Process ID	Arrival Time	Burst Time	Priority
1	0	25	10
2	1	2	1
3	2	2	1
4	3	2	1
5	4	2	1
6	5	2	1
7	6	2	1
8	7	2	1

Round Robin | Quantum = 2

P1

P2

P3

P1

P4

P5

P6

P7

P8

P1

P1

P1

P1

P1

P1

0246810121416182022242628

Queue: P1 -> P2 -> P3 -> P1 -> P4 -> P5 -> P6 -> P7 -> P8 -> P1

Process	Arrival	Burst	Priority	WT	TAT	RT
P1	0	25	10	14	39	0
P2	1	2	1	1	3	1
P3	2	2	1	2	4	2
P4	3	2	1	5	7	5
P5	4	2	1	6	8	6
P6	5	2	1	7	9	7
P7	6	2	1	8	10	8

Averages: WT = 6.50 TAT = 11.38 RT = 4.75

Avg WT: 6.50 | Avg TAT: 11.38 | Avg RT: 4.75

Preemptive Priority Scheduling

P1

P2

P2

P3

P3

P4

P4

P5

P5

P6

P6

P7

P7

P8

P8

01234567891011121314

Queue: P1 -> P2 -> P3 -> P4 -> P5 -> P6 -> P7 -> P8 -> P1

Process	Arrival	Burst	Priority	WT	TAT	RT
P1	0	25	10	14	39	0
P2	1	2	1	0	2	0
P3	2	2	1	1	3	1
P4	3	2	1	2	4	2
P5	4	2	1	3	5	3
P6	5	2	1	4	6	4
P7	6	2	1	5	7	5

Averages: WT = 4.38 TAT = 9.25 RT = 2.63

Avg WT: 4.38 | Avg TAT: 9.25 | Avg RT: 2.63

Scenario E — Validation Demo

Scenario E does not run a simulation. Its sole purpose is to demonstrate that the input validation layer correctly rejects malformed data before any scheduling logic executes.

Invalid Input Submitted

Violation	Error Message Produced
Violation 1	Duplicate process ID detected: P1 appears twice. All process IDs must be unique.
Violation 2	Process P1 (second entry): burst time is 0. Burst time must be at least 1.
Violation 3	Process P1 (second entry): arrival time is 0. Arrival time must be at least 0.

Validation coverage: Duplicate IDs · Negative arrival time · Zero burst time · Negative priority · Invalid quantum. The simulation panel displays all collected errors in red before any algorithm runs. The validator is implemented in InputValidator.java and is called from SimulationPanel.runSimulation() on every run.

CPU Scheduling Comparator — Round Robin vs Priority

SimulationComparison

Time Quantum: 2 Priority Rule: ☒ Lower number = higher priority ☐ Higher number = higher priority Preset: Scenario E - Validation demo Load Scenario

Add ProcessQuick PasteDelete ItemClear AllRun SimulationReset

- Two processes share the ID 1. All process IDs must be unique.
- Process P1 has a burst time of 0. Burst time must be at least 1.
- Process P1 has an arrival time of -1. Arrival time must be 0 or more.

Process Input Table

Process ID	Arrival Time	Burst Time	Priority
1	1	0	4
1	1	-1	0

4. Cross-Scenario Comparison

The table below summaries which algorithm performed better across all four runnable scenarios. Scenario E is excluded as it produces no metrics.

Metric	Scenario A	Scenario B	Scenario C	Scenario D
Avg Waiting Time winner	Priority	Priority	Priority	Priority
Avg Turnaround winner	Priority	Priority	Priority	Priority
Avg Response Time winner	Round Robin	Round Robin	Round Robin	Priority
Starvation risk observed	No	Yes (P5)	Yes (P1)	Yes (P1)

4.1 Key Observations

Priority scheduling wins on average waiting and turnaround time in every scenario where priorities differ meaningfully. This is expected: by always running the most urgent job first, priority scheduling minimises the total time high-value jobs spend waiting, which pulls down the average.

Round Robin wins on average response time in every scenario. This is also expected and is the defining property of time-sliced scheduling: because the CPU rotates among all ready processes, every process receives its first CPU slice quickly. No process waits for all others to complete before it gets a turn.

Starvation risk appears in Scenarios B, C, and D under Priority Scheduling. In all three cases, at least one low-priority process waits far longer than the average — in Scenario C, P1 waits 16 time units simply because four higher-priority processes arrived simultaneously. Round Robin completely avoids this: its round-trip guarantee ensures every process gets CPU time in proportion to the number of ready processes and the quantum size.

5. Conclusion

5.1 Algorithm Trade-off Summary

Property	Round Robin	Preemptive Priority
Fairness	High — every process rotates regularly	Low — priority controls access order
Starvation risk	None — all processes advance	Yes — low-priority jobs can be blocked
Response time	Excellent — bounded by $(n-1) * q$	Varies — urgent jobs fast, others slow
Urgency handling	None — all jobs treated equally	Excellent — critical jobs preempt immediately
Avg waiting time	Moderate	Lower when priorities are spread out
Use case	Interactive / time-sharing systems	Real-time / mission-critical systems

5.2 Recommendation

The correct choice between Round Robin and Preemptive Priority Scheduling depends entirely on the requirements of the target system, not on which algorithm wins across an arbitrary set of benchmarks.

Round Robin is the right choice when all processes are considered equally important and interactive response time matters. Time-sharing operating systems, desktop environments, and web servers handling concurrent requests benefit from Round Robin's guarantee that no request will be ignored for long.

Preemptive Priority Scheduling is the right choice when some processes are genuinely more important than others and urgency must be reflected in scheduling decisions. Real-time systems, operating system kernel threads, network interrupt handlers, and safety-critical embedded systems are examples where low-priority jobs can and should wait while high-priority jobs run.

A hybrid approach — assigning priorities to process classes and using Round Robin within each priority class — is common in production operating systems (including Linux's Completely Fair Scheduler and Windows thread priorities) and represents a practical synthesis of both algorithms studied here.